CS 61A
Past Exam Practice

Efficiency and Gleam Data - Solutions

Solutions last updated: Sunday, November 8, 2026

Your Name: _____

Your Student ID: _____

Room you are taking the exam in: _____

Name and SID of the person to your **left**: _____

Name and SID of the person to your **right**: _____

You have 110 minutes. There are 5 questions of varying credit. (19 points total)

Question	1	2	3	4	5	Total
Points	3	2	8	5	1	19

For questions with **circular bubbles**, you may select only one choice.

- ○ Unselected option (Completely unfilled)
- ◎ Don't do this (it will be graded as incorrect)
- ● Only one selected option (completely filled)

For questions with **square checkboxes**, you may select one or more choices.

- ■ You can select
- ■ multiple squares
- ☑ (Don't do this)

Make sure to **write your student ID at the top of each page** in the provided blank.

Anything you write outside the answer boxes or you ~~cross out~~ will not be graded. If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade the worst interpretation. For coding questions with blanks, you may write at most one statement per blank and you may not use more blanks than provided.

As a member of the UC Berkeley community, I act with honesty, integrity, and respect for others. I will follow the rules of this exam. I will not use any other unauthorized materials or devices during this exam. I will not communicate with anyone other than the proctors during this exam. I will not share any information about the exam with anyone until after the end of the exam period.

Acknowledge that you have read and agree to the honor code above and **sign your name below**:

_____

This content is protected and may not be shared, uploaded, or distributed.

Q1 *Efficiency Practice* **(3 points)**

Adapted From: Spring 2026 Midterm 2 - Question 9 | **Level:** Regular

Both functions keep only the words shorter than `length`. Each question asks for one count for an input linked list containing `n` words. For loops, count how many times the body runs; do not count condition checks.

Recursive version.

```
1  def remove_long_words1(links, length):
2      if not isinstance(links, Link):
3          return ()
4      if len(links.first) >= length:  # A
5          return remove_long_words1(links.rest, length)
6      return Link(links.first, remove_long_words1(links.rest, length))
```

Q1.1 (1 point) For an input list of `n` words, how many times does line A run in total?

n times

Iterative version. This version can change the links in the input.

```
1  def remove_long_words2(links, length):
2      head = links
3      while isinstance(head, Link) and len(head.first) >= length:  # Loop 1
4          head = head.rest
5      if not isinstance(head, Link):
6          return ()
7      prev = head
8      curr = head.rest
9      while isinstance(curr, Link):  # Loop 2
10         if len(curr.first) >= length:
11             prev.rest = curr.rest
12         else:
13             prev = curr
14         curr = curr.rest
15     return head
```

Q1.2 (1 point) Suppose the input contains `n >= 1` words and the first word is shorter than `length`. How many times does the body of Loop 2 run?

n - 1 times

Q1.3 (1 point) Now suppose all `n` words have length greater than or equal to `length`. How many times does the body of Loop 1 run?

n times

Solution: When the first word is short, Loop 1 is skipped and `curr` starts at the second word. Loop 2 visits each of the remaining `n - 1` words once, including words it removes. When every word is long, Loop 1 advances through all `n` words before reaching the empty list.

Gleam Reference

This page summarizes the Gleam features used in this worksheet. Types are capitalized, function bodies end with the expression they return, and values are immutable.

Functions and types

```
pub fn square(x: Int) -> Int {
  x * x
}

pub fn twice(f: fn(a) -> a, x: a) -> a {
  f(f(x))
}
```

`a` is a type variable. For example, `List(a)` means a list whose elements all have the same type, whatever that type is.

Anonymous functions use `fn`:

```
fn(x) { x + 1 }
```

Define named recursive helpers at the top level, not inside another function. A recursive call is tail recursive when its result is returned directly, with no remaining operation.

Operators

- Arithmetic: `+`, `-`, `*`, `/`, `%`
- Comparison: `==`, `!=`, `<`, `<=`, `>`, `>=`
- Boolean: `&&`, `||`, `!`
- Strings: `<>` joins two strings

Custom types and records

```
pub type Shape {
  Circle(name: String, radius: Int)
  Square(name: String, side: Int)
}

pub type Box(a) {
  Box(label: String, item: a)
}
```

`Circle` and `Square` are the variants of `Shape`. Call a variant to make a value, such as `Square("tile", 2)`.

`Box(a)` has a type variable `a`, so `Box("n", 7)` has type `Box(Int)`.

`shape.name` works because every variant has `name` in the same position with the same type. For other fields, use `case`:

```
case shape {
  Circle(_, r) -> 2 * r
  Square(_, s) -> s
}
```

In a pattern, `_` ignores one field, and `..` ignores the rest, as in `Square(..)`.

`Box(..b, item: 8)` is a record update: a new `Box` with the same fields as `b` except `item`. `b` itself is unchanged.

Case expressions and patterns

```
case n {
  0 -> "zero"
  _ if n < 0 -> "negative"
  _ -> "positive"
}
```

A `case` can match multiple subjects: `case x, y { ... }`. The `_` pattern matches any value, and `if` adds a guard to a pattern.

Lists

```
case s {
  [] -> []
  [first, ..rest] -> [first, ..rest]
  [a, b, ..tail] -> [a, b, ..tail]
}
```

- `[]` is empty.
- `[first, ..rest]` splits a non-empty list.
- `[first, ..rest]` also constructs a new list.
- `#(x, y)` is a two-element tuple.

Useful standard-library functions:

- `list.append(xs, ys)` joins two lists.
- `list.map(xs, f)` applies `f` to each value.
- `list.filter(xs, keep)` keeps values satisfying `keep`.
- `list.length(xs)` returns a list's length.
- `int.absolute_value(n)` returns an integer's absolute value.
- `int.to_string(n)` converts an integer to a string.

Import their modules with `import gleam/list` or `import gleam/int`.

Results and dicts

`Result(value, error)` has two variants: `Ok(value)` for success and `Error(error)` for failure. `Nil` is a value that holds no information.

`import gleam/dict.{type Dict}` lets you write `Dict(k, v)`, a dict with keys of type `k` and values of type `v`. Dicts are immutable.

- `dict.new()` is an empty dict.
- `dict.insert(d, k, v)` returns a new dict where `k` maps to `v`, and contains all the key value pairs of dict d.
- `dict.get(d, k)` returns `Ok(v)` if `k` maps to `v`, or `Error(Nil)` if `k` is not a key.

Q2 *No More Folders!!* **(2 points)**

Level: Regular

An `Entry` value is either a `File` containing its name or a `Folder` containing a list of entries. Implement `list_files`, which takes in a list of Entry records `entries` and returns a list of strings which are the file names of every file in `entries`.

```
import gleam/list

pub type Entry {
  File(name: String)
  Folder(contents: List(Entry))
}

pub fn list_files(entries: List(Entry)) -> List(String) {
  case entries {
    [] -> []
    [Folder(contents), ..rest] ->
      list.append(_______, _______)             // (a), (b)
    [File(name), ..rest] ->
      [_______, .._______]                      // (c), (d)
  }
}

let entry_list0 = [
  Folder([File("Hello.txt")]),
  File("World.txt")
]

assert list_files(entry_list0) == ["Hello.txt", "World.txt"]

let entry_list1 = [
  Folder([Folder([File("a.txt")]), File("b.txt"), File("c.txt")]),
  File("d.txt"),
  Folder([File("e.txt"), Folder([File("f.txt")])])
]

assert list_files(entry_list1) == ["a.txt", "b.txt", "c.txt", "d.txt", "e.txt", "f.txt"]

let entry_list2 = [
  Folder([File("a.txt"), Folder([File("b.txt"), File("c.txt")])]),
  Folder([]),
  File("d.txt")
]

assert list_files(entry_list2) == ["a.txt", "b.txt", "c.txt", "d.txt"]
```

Q2.1 (1 point) Fill in blanks (a) and (b).

(a) `list_files(contents)`

(b) `list_files(rest)`

Q2.2 (1 point) Fill in blanks (c) and (d).

(c) `name`

(d) `list_files(rest)`

Q3 *Online Learning Site* **(8 points)**

Adapted from: Spring 2021 - Midterm 2 - Question 5 | **Level:** Regular-Hard

An online learning site is described by the `LearningSite` record, which keeps track of the following information:

- `catalog`: a dictionary of the published content, keyed by title
- `watched`: a list of `Content` records that students have opened, in the order they opened them. The first element is the first content viewed.
- `missing`: a list of titles that students asked for but could not be found. The most recently searched title should be at the front of the list.

Both exercises and videos have a title and an author, so they share a `Details` record. Additionally, the `watched` and `missing` lists may contain duplicates if content is viewed or searched more than once.

```
import gleam/dict.{type Dict}
import gleam/int
import gleam/list

pub type Details {
  Details(title: String, author: String)
}

pub type Content {
  Exercise(details: Details, num_questions: Int)
  Video(details: Details, num_seconds: Int)
}

pub type LearningSite {
  LearningSite(
    catalog: Dict(String, Content),
    watched: List(Content),
    missing: List(String),
  )
}

pub fn description(content: Content) -> String {
  case content {
    Exercise(details, _) -> details.title <> " by " <> details.author
    Video(details, seconds) ->
      details.title <> " by " <> details.author
      <> " (" <> int.to_string(seconds) <> " seconds)"
  }
}
```

Implement the following functions for the online learning site:

- `fn publish(s: LearningSite, content: Content) -> LearningSite`: adds `content` to the `LearningSite` catalog
- `fn view(s: LearningSite, title: String) -> LearningSite`: a student views the content with `title`. If the catalog has this content, add the content object to the end of the watched list. If not, add the title to the front of the missing list.
- `fn watch_list(s: LearningSite) -> List(String)`: return the description of each watched content, in the order watched.
- `fn count_videos(s: LearningSite) -> Int`: return how many of the watched contents are videos. A video that was watched twice counts twice.

```
let cs61a = LearningSite(dict.new(), [], [])
let recursion = Exercise(Details("Recursion", "Chris D"), 5)
let efficiency = Video(Details("Efficiency", "Kay O"), 881)

let cs61a = publish(cs61a, recursion)
let cs61a = publish(cs61a, efficiency)
assert dict.get(cs61a.catalog, "Recursion") == Ok(recursion)

let cs61a = view(cs61a, "Efficiency")
let cs61a = view(cs61a, "LinkedLists")
let cs61a = view(cs61a, "Efficiency")
let cs61a = view(cs61a, "Recursion")
let cs61a = view(cs61a, "Mutability")
let cs61a = view(cs61a, "Efficiency")
assert cs61a.watched == [efficiency, efficiency, recursion, efficiency]
assert cs61a.missing == ["Mutability", "LinkedLists"]

assert watch_list(cs61a)
  == [
    "Efficiency by Kay O (881 seconds)",
    "Efficiency by Kay O (881 seconds)",
    "Recursion by Chris D",
    "Efficiency by Kay O (881 seconds)"
  ]

assert count_videos(cs61a) == 3
```

```
pub fn publish(s: LearningSite, content: Content) -> LearningSite {
  LearningSite(..s, catalog: _______)                    // (a)
}

pub fn view(s: LearningSite, title: String) -> LearningSite {
  case _______ {                                         // (b)
    Ok(content) -> _______                               // (c)
    Error(Nil) -> _______                                // (d)
  }
}

pub fn watch_list(s: LearningSite) -> List(String) {
  _______                                                // (e)
}

pub fn count_videos(s: LearningSite) -> Int {
  list.length(
    list._______(_______, _______ {                      // (f), (g), (h)
      case _______ {                                     // (i)
        _______                                          // (j)
        _______                                          // (k)
      }
    }),
  )
}
```

Q3.1 (1 point) Fill in blank (a).

```
dict.insert(s.catalog, content.details.title, content)
```

Q3.2 (1 point) Fill in blank (b).

```
dict.get(s.catalog, title)
```

Q3.3 (1 point) Fill in blank (c).

```
LearningSite(..s, watched: list.append(s.watched, [content]))
```

Q3.4 (1 point) Fill in blank (d).

```
LearningSite(..s, missing: [title, ..s.missing])
```

Q3.5 (1 point) Fill in blank (e).

```
list.map(s.watched, description)
```

Q3.6 (1 point) Fill in blanks (f), (g), and (h).

(f) `filter`

(g) `s.watched`

(h) `fn(content)`

Q3.7 (1 point) Fill in blank (i).

```
content
```

Q3.8 (1 point) Fill in blanks (j) and (k).

(j) `Video(_, _) -> True`

(k) `Exercise(_, _) -> False`

Q4 *Making Pen Friends!* **(5 points)**

Adapted from: Spring 2026 - Midterm 2 - Question 5 | **Level:** Challenge

A `Pen` has a name and a dict that maps each animal's name to its `Animal` record. An `Animal` has a unique name and a list of the names of its friends. Two animals may only become friends if they are in the same pen. When they do, each one's name is added to the front of the other's list of friends.

Implement `add_animal`, which returns a new pen that also holds an animal named `name` with no friends. Then implement `befriend`, which returns `Ok` of a new pen in which the animals named `a` and `b` are friends, or an `Error` if either animal is not in `pen`.

```
1  import gleam/dict.{type Dict}
2
3  pub type Animal {
4    Animal(name: String, friends: List(String))
5  }
6
7  pub type Pen {
8    Pen(name: String, animals: Dict(String, Animal))
9  }
10
11 pub fn add_animal(pen: Pen, name: String) -> Pen {
12   Pen(..pen, animals: _______)                                    // (a)
13 }
14
15 pub fn befriend(pen: Pen, a: String, b: String) -> Result(Pen, String) {
16   case _______, _______ {                                         // (b), (c)
17     Ok(x), Ok(y) -> {
18       let new_x = Animal(..x, friends: _______)                   // (d)
19       let new_y = Animal(..y, friends: [a, ..y.friends])
20       Ok(Pen(..pen, animals: _______))                            // (e)
21     }
22     _, _ -> Error("Cannot befriend an animal that doesn't share a pen!")
23   }
24 }
25
26 let my_pen = add_animal(add_animal(Pen("coop", dict.new()), "Alice"), "Clarice")
27 let other_pen = add_animal(Pen("barn", dict.new()), "Bob")
28
29 assert dict.get(other_pen.animals, "Bob") == Ok(Animal("Bob", []))
30
31 assert befriend(my_pen, "Clarice", "Bob")
32   == Error("Cannot befriend an animal that doesn't share a pen!")
33
34 assert befriend(my_pen, "Clarice", "Alice")
35   == Ok(Pen("coop", dict.from_list([
36     #("Alice", Animal("Alice", ["Clarice"])),
37     #("Clarice", Animal("Clarice", ["Alice"])),
38   ])))
39
```

Q4.1 (1 point) Fill in blank (a).

```
dict.insert(pen.animals, name, Animal(name, []))
```

Q4.2 (1 point) Fill in blanks (b) and (c).

(b)
```
dict.get(pen.animals, a)
```

(c)
```
dict.get(pen.animals, b)
```

Q4.3 (1 point) Fill in blank (d).

```
[b, ..x.friends]
```

Q4.4 (1 point) Fill in blank (e).

```
dict.insert(dict.insert(pen.animals, a, new_x), b, new_y)
```

Q4.5 (1 point) After `befriend(my_pen, "Clarice", "Alice")` in the doctests above, what does `dict.get(my_pen.animals, "Alice")` evaluate to?

Recall that Gleam records are immuable. The my_pen object is therefore unchanged from the befriend call mentioned in the question. The answer is `Ok(Animal("Alice", []))`

Q5 *What Would Gleam Do?* **(1 points)**

Level: Challenge

Assume the following code has been evaluated:

```
pub fn shrink(k: Int, t: List(Int)) -> fn(List(Int)) -> List(Int) {
  fn(s) {
    case s {
      [] -> t
      [first, ..rest] if first == k -> {
        let next = shrink(k + 1, t)
        next(rest)
      }
      [first, ..rest] -> {
        let next = shrink(k, [first, ..t])
        next(rest)
      }
    }
  }
}
```

Q5.1 (1 point) What does the final expression evaluate to?

```
let run = shrink(3, [])
run([3, 1, 4, 1, 5, 9, 2, 6])
```

```
[2, 9, 1, 1]
```